Tools and Held-Out Tasks for Testing Agent-Written Weight Edits on Gemma 4

weightprog: closed-form editing with matched-control checks; 186 offline held-out agent tasks and a 320-call replay set

Abstract

A weight edit that fixes one behaviour can break others, and measuring the target alone hides that cost. We release tools and data for measuring both when a coding agent edits Gemma 4. `weightprog` gives the agent instruments to find where the model makes a choice, constructors for low-rank closed-form edits, and an evaluation against the base model and random edits of the same size that reports target gains, preservation losses and missing evidence together. OOD-C packages 186 SWE-rebench tasks from held-out repositories to run offline in the Gemma 4 Developer Agent Competition's harness, and the replay set holds 320 recorded edit calls. In a comparison fixed in advance on a new target, Claude Opus 5.5 built edits that reduced target errors with and without the library, but neither kept every unrelated output unchanged, and random edits often failed that check too. One run with and one without cannot settle whether the library helps.

1. Introduction

Small open models used as coding agents have habits their users want changed: an outdated API, a tool-call mistake, a formatting choice. Frontier models can now write weight edits that change such habits [1]. An agent doing this needs instruments to find the behaviour, ways to build an update, and a test that tells a targeted improvement from damage elsewhere.

Our companion study shows why the test matters [2]. Its subject is Gemma 4 E4B, a small open model working as a coding agent in the harness of the Gemma 4 Developer Agent Competition on Kaggle. When E4B edits a file, it often copies escape codes such as `\n` from the tool's escaped output into the *old text*, the passage its edit should replace. The study's *gate* was a test fixed in advance, or *registered*: its plan was time-stamped in the study's log before the run. On 320 recorded edit calls from held-out repositories, three edits written by frontier models (the *editors*) cut these escape artefacts beyond random edits of the same size. But each lost 3 to 11 of the 213 *working calls*, those whose old text the unedited model found, in its worse generation run (*draw*; the gate had two), where the rule allowed 2 per draw.

We present three resources (Figure 1). `weightprog` starts from behaviour cases and produces edits and comparisons. OOD-C starts from software issues, for whole-agent runs offline. The replay set starts from recorded tool calls, for comparing a model's continuation under fixed context; a study kit scores edits on it. Each can be used alone.



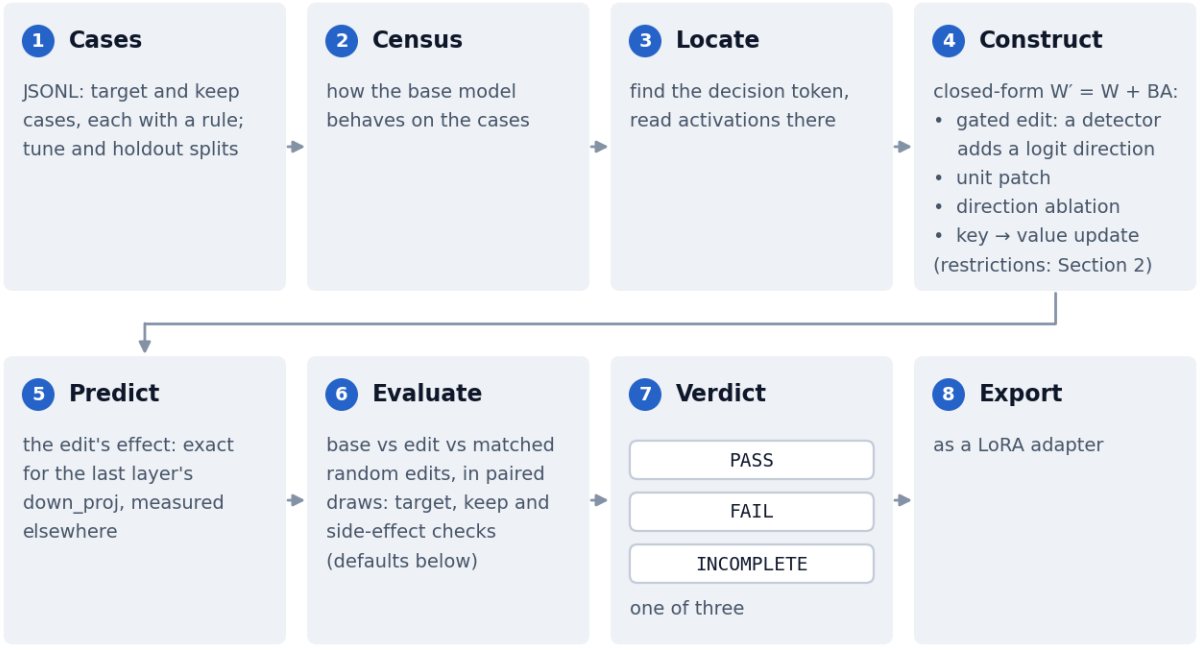
Figure 1. The three resources and the study kit, with their entry points. "Regression fixture": the library's tests reproduce stored results from the replay set.

2. weightprog

A *case* is a prompt and a rule for judging its output. *Target cases* say what should improve, *keep cases* what should survive. A rule either sorts a generated output into good, bad or other with text patterns or a checking function (such cases are *judged*, and targets of this kind *generated*), or compares the log-probabilities of two given continuations without generating (*contrast* targets). A keep case without a rule must keep its greedy output unchanged. Each case has a group, such as a repository, and a split, tune or holdout; all cases of a group share a split, so an edit is judged on groups it was not built from.

The workflow starts by measuring the base model (Figure 2). In the spelling example, target cases ask for US spelling and keep cases ask for British spelling or unrelated answers. A *census* generates the base model's outputs on the tune cases and counts good, bad and other. The agent then finds the *decision token*, where the two spellings diverge, reads activations there, builds a candidate edit and evaluates it:

```
weightprog census --model-dir MODEL --cases cases.jsonl --split tune
weightprog eval --model-dir MODEL --cases cases.jsonl --split holdout --edit edit/ --controls 2 --side-effec
```



Illustrative case format the library's spelling example (its run on E4B: Section 4)

1 **target** `"In US English, write one sentence describing the shade of a barn."`
good `\bcolor` **bad** `\bcolour`

keep `"In British English, write one sentence describing the shade of a barn."`
good `\bcolour` **bad** `\bcolor\b`

keep `"List three prime numbers."`
no rule: the output should not change

3 the decision token: the first token at which " color" and " colour" differ

Default checks step 6; users can change them, and should fix them before the holdout run

Random edits	two by default; each changes the same modules with the same rank and the same Frobenius norm of BA per module, in a Gaussian random direction
Target	the bad rate (generated targets) or the log-probability margin (contrast targets) improves against the base and against every random edit in one-sided sign tests over cases, $p < 0.05$, and a 95 % bootstrap interval over groups for the gain against the base lies above zero; for generated targets, good outputs also rise by at least half the fall in bad ones
Keep	at most 2 keep cases with a rule turn from good to not good per draw, and at most 10 % of the rule-less keep cases change their greedy output; the random edits' own losses are shown beside, not used
Side effects	on reference texts kept apart from fitting: mean KL(base ‖ edit) at most 0.05 and mean increase in negative log-likelihood (NLL) at most 0.02, in nats per token
Verdict	PASS when every check was measured and passed; FAIL when a check failed; INCOMPLETE when one is missing

Figure 2. The workflow, the case format of the library's spelling example (illustrative), and the default checks of step 6. In the rules, `\b` marks a word boundary.

An edit adds a low-rank change BA to chosen weight matrices W, fitted in closed form from activations, without gradients. The constructors include a *gated edit*, a detector on MLP activations that adds a direction in token-score

(logit) space when it fires, close to Transformer-Patcher [21]; *unit patching*, which rewires MLP units the model rarely uses [21, 22]; *direction ablation*, which removes a direction from a layer's output [23]; key-to-value updates as in ROME and MEMIT [3, 4]; and a fixed residual-stream direction, that is, activation steering [24] written into the weights. Two optional restrictions limit what an update does to inputs that should keep their behaviour. *Covariance weighting* [3, 4] penalises change along the directions those inputs' activations use, in proportion to their second moment; *null-space restriction* [5] restricts the update to directions those activations barely use. For the last layer's down-projection, the library predicts an edit's effect on token scores exactly from captured activations; elsewhere it runs the model.

Evaluation is the centre of the library. It generates outputs for the base model, the edit and random edits matched in modules, rank and size, in paired draws (by default one greedy, one sampled), and records good, bad and other outputs separately. A target counts as improved only if it beats the base and every random edit across cases, and good outputs must also rise, so turning bad output into garbage does not count. Keep cases may lose only a fixed number of good outputs, and side effects on held-apart texts are measured by KL divergence and the change in negative log-likelihood (NLL). Figure 2 gives the default thresholds; users can change them and should fix them before the holdout run. The verdict is PASS, FAIL or INCOMPLETE, the last when a required check was not measured.

An edit is exported as a PEFT LoRA adapter [6] whose change equals BA exactly; export does not certify behaviour.

3. Design choices and method coverage

We chose what to include by mapping editing and interpretability methods against what the nine editing runs of the companion study used (Figure 3, with a key to the runs). Eight runs had a frontier model as the editor; one, the *recipe*, was a fixed procedure of ours. The study ran in two rounds: round 1 aimed at whether the old text is found at all, round 2 at escape artefacts. No run used activation patching, covariance weighting, null-space restriction or sparse autoencoders. All three edits submitted to the gate in round 2 lost more working calls than allowed, so we added activation patching, to locate where a behaviour is computed, a constructor for the unit patches Opus r2 had built by hand, and both restrictions. The mapping is post hoc and covers only our nine runs.

Method family	weightprog support	Validated on	Used in the companion study's runs (key below)
Find the behaviour			
Linear probes, read-outs [14]	✓ yes	○ tiny models only	• all 9 runs
Logit lens, direct logit attribution [15, 16]	✓ yes	○ tiny models only	• Fable r2, Astra r2, Opus inf
Activation capture at decision tokens [10–12]	✓ yes	● **real E4B**	• all 9 runs
Difference of means [17]	✓ yes	○ tiny models only	• recipe r1, Opus r1
Exact prediction of an edit's effect (§2)	✓ yes	● **real E4B**	• Fable r2, Opus inf, a Fable 5.2 run
Activation patching [3, 18]	✓ yes	○ tiny models only	○ none
Attention and head analysis [19]	~ part	○ tiny models only	• Fable r1
Sparse autoencoders [20]	✗ no	—	○ none
Change it			
Gated logit-direction edit (§2; cf. [21])	✓ yes	○ tiny models only	• Fable r2, Astra r2, Fable 5.2 rep
Unit (neuron) patching [21, 22]	✓ yes	● **real E4B**	• Opus r2, Opus inf, Fable 5.2 inf
Key → value update with covariance [3, 4]	✓ yes	○ tiny models only	○ none
Null-space restriction [5]	✓ yes	○ tiny models only	○ none
Directional ablation [23]	✓ yes	○ tiny models only	• recipe r1, Fable 5.2 rep
Steering written into weights [24]	✓ yes	○ tiny models only	• Opus inf
Attention-side edits (§3)	~ part	○ tiny models only	• Fable r1, Opus r1
Scaling and merging edits [25]	✓ yes	○ tiny models only	• Fable 5.2 rep
Ship it			
Export as a LoRA adapter [6]	✓ yes	● **real E4B**	—

✓ a function and docs ~ building blocks only ✗ not in the library

● checked on the real E4B model ○ tested only on tiny random Gemma 4 models (Section 4)

Runs of the companion study

r1	round 1: is the old text found at all?	**recipe**	our scripted difference-of-means ablation
r2	round 2: escape artefacts	**inf**	informed: read the earlier runs' records
Opus	Claude Opus 5.5	**rep**	repair: began from a gradient-trained adapter
Fable	Claude Fable 5.1; 5.2 where marked	**round 1**	Opus, Fable, recipe
Astra	GPT-6 Astra	**round 2**	Opus, Fable, Astra, Opus inf, Fable 5.2 inf, Fable 5.2 rep
no edit	Astra r2; Fable 5.2 inf and rep withdrew		

Figure 3. Method families (with a reference or section), library support, validation level, and the companion study's runs that used them.

The restrictions limit interference; they do not remove it. Both aim to limit changes on reference inputs at the edited layer, so keep cases must still be measured. Attention-side edits can target the attention output projection but not

queries, keys or values, and the library has no sparse-autoencoder dictionaries.

4. Validation and a worked use

Validation is stated per operation (Table 1).

Table 1. Validation evidence for `weightprog` 0.2.0.

Evidence	What it checks
94 offline tests	Tiny random models, both backends (PyTorch and MLX), the quickstart and example code
Five checks on real E4B	Base generation, PyTorch against MLX, exact prediction of a last-layer edit, captured activations, reproduction of the companion study's gate
Adapters served	The export helper, through vLLM, including the competition's scorer

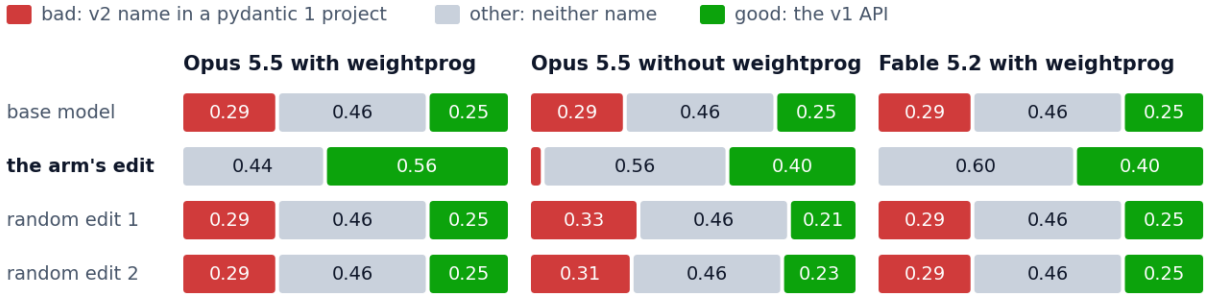
The tiny models are randomly initialised Gemma 4 checkpoints with E4B's structure at toy size (6 layers, width 64); they check that the code runs and computes what it should, not that an edit works. Most methods are so far tested only on them (Figure 3). The spelling example also ran on E4B, but E4B wrote neither spelling on its prompts, so the run checks only the code path. A public Kaggle notebook runs the library on E4B: it shows token by token where the companion study's escape edit acts, as in that study's Figure 5, and fits and exports a rank-1 edit.

To test whether the library helps an agent with a new target, a comparison fixed in advance gave two Claude Opus 5.5 runs (*arms*), one with the library and one without, four hours each in the same sandbox at the same effort setting. The target: keep the pydantic 1 API in projects pinned to pydantic 1, where E4B writes pydantic 2 names that do not exist there, such as `.model_json_schema()` for `.schema()`, while pydantic 2 projects and unrelated code stay unchanged. A base-model census set this direction: in pydantic 1 projects E4B wrote pydantic 2 names for `schema`, `fields` and `validate_json`, three APIs chosen by a rule written beforehand, at rates of 0.81, 0.53 and 0.16, and never the reverse. Then we froze the plan. The arms differ only in a paragraph naming the library and in its installation. A judge script fixed in advance applies the default checks to held-out cases with other wording and domains (Figure 4). A possible bias toward the library: its author wrote the documentation, cases and judge knowing the target, and assigned targets and keeps after the census; its worked examples use other targets.

Neither arm passed (Figure 4). Both low-rank MLP edits cut the target's bad rate, the share of outputs using a pydantic 2 name, from 0.29 to 0.00 with the library and to 0.04 without it, broke no keep case with a rule, and passed the side-effect check. Both failed keep because unrelated outputs changed: one of four with the library, as with each of its random edits, and two without. A follow-up, Claude Fable 5.2 with the library, registered after the first arm ended but before judging, to run unless both passed, was judged separately and failed similarly. All three edits passed on the tune cases.

With four prompts the keep limit allowed no changes: 35 of 60 random edits of the same size, sampled afterwards, failed it too (Figure 4c). On 48 plain-code prompts (4 holdout, 4 tune, 40 new), only one of the 40 random edits for the two library edits changed as few outputs as they did; the edit without the library changed about as many as its random edits. Future tests should use more prompts and calibrate the limit against random edits on development data. The registered verdicts stand, and one run per arm cannot separate the library's effect from chance.

(a) Outputs on the 24 held-out target cases, two draws each



(b) The three registered checks on the holdout; an edit passes only if all three pass

	Opus, with weightprog	*Opus, without*	*Fable 5.2, with (follow-up)*
Target ✓ **pass, all three**	$p \le 0.004$ against the base and both random edits	$p \le 0.008$ against the base and both random edits	$p \le 0.004$ against the base and both random edits
Keep ✗ **fail, all three**	with a rule, broken: 0 unrelated, changed: 1 of 4 (random edits: 1 and 1)	with a rule, broken: 0 unrelated, changed: 2 of 4 (random edits: 0 and 0)	with a rule, broken: 0 unrelated, changed: 1 of 4 (random edits: 1 and 0)
Side effects ✓ **pass, all three**	KL 0.00007 nats/token (random 0.00058, 0.00056)	KL 0.00066 nats/token (random 0.00109, 0.00102)	KL 0.00004 nats/token (random 0.00083, 0.00076)

Holdout: 54 cases in domains not used for tuning: 24 target cases, 24 pydantic 2 keep cases and 6 unrelated keep cases, 4 of them without a rule. Keep passes if at most 2 keep cases with a rule break per draw and at most 10 % of the 4 unrelated prompts without a rule change their greedy output; with 4 prompts, none may change. Side effects: mean KL at most 0.05 and mean NLL increase at most 0.02 nats per token, on 16 texts neither arm saw.

(c) After judging, post hoc: unrelated outputs that change on 48 plain-code prompts

each arm's edit against 20 new random edits of its size; the 48 are the 4 holdout, 4 tune and 40 new prompts; changed: the greedy output differs from the base model's; fail keep: changes any of the 4 holdout prompts

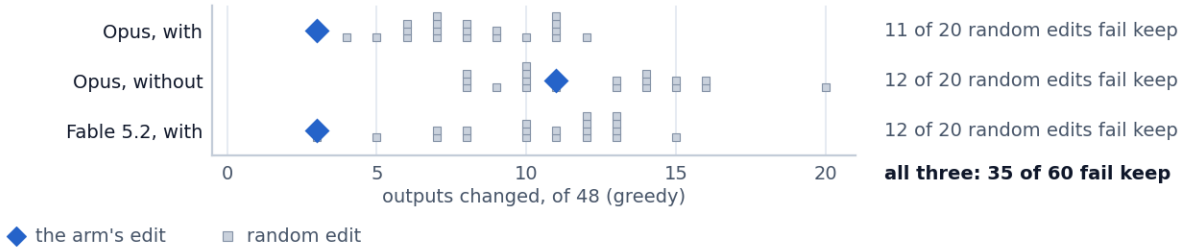


Figure 4. The usability test: (a) target outputs on the held-out cases; (b) the three registered checks, all of which an edit must pass; (c) post hoc, how many unrelated outputs each edit and its random edits change.

5. OOD-C

OOD-C makes 186 tasks from SWE-rebench [7, 8] usable as held-out validation in the competition's harness (Figure 5). The tasks, real issues with their reference fixes and tests from 148 Python repositories, are SWE-rebench's (CC-BY-4.0). We add:

- Selection: no repository is shared with the competition's four training repositories or the companion study's two earlier held-out sets (OOD: out of distribution; C: the third set). Issues date from 2025 on, with at most two tasks per repository and OSI-approved licences only.
- Offline environments: the harness runs a task without network access, so each task has a repository snapshot at its base commit and package sets for Python 3.13, as in Kaggle's scorer image, and 3.12, as on Colab.
- Checks: a task is kept when our E4B agent runs on it in the harness without error, driven by a scripted stand-in for the model. Gradability is a separate flag: in 60 tasks the *gold fix*, the reference patch from the original pull request, makes the tests pass twice and the empty patch does not, and 53 of these pass again under both Python

versions. The companion study counted solved tasks on these 53 and escape artefacts on all 186.
- Provenance: each task's source row and licence, and build and validation reports for every candidate.

OOD-C is thus a validated packaging of existing tasks for one harness, not a new benchmark. Held-out does not mean absent from pretraining, and issue texts were not audited for hints. When the harness runs without its container sandbox, an agent could read the gold patches, so solution-rate evaluations should keep them out of reach.

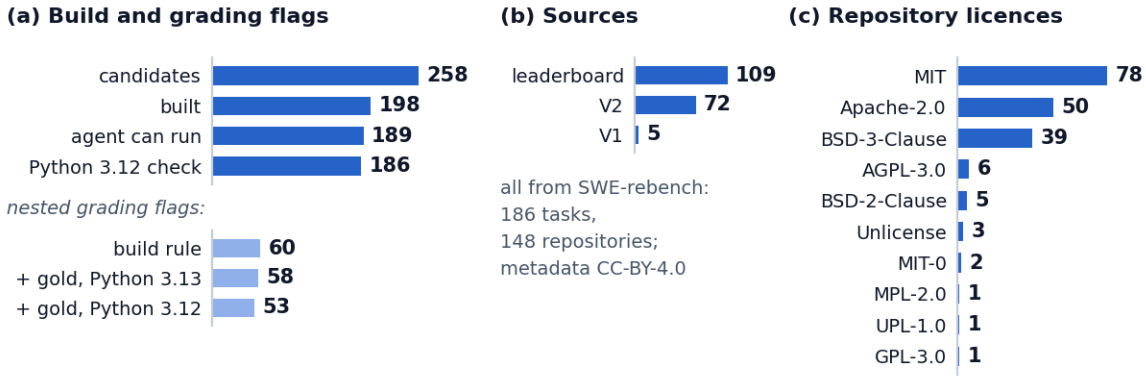


Figure 5. OOD-C: (a) from candidates to the set, and the nested grading flags; (b) sources within SWE-rebench; (c) repository licences, counted by task.

6. The replay test set

An `edit_file` call names a file, an *old text* to find and a new text to put in its place. The replay set holds 320 such calls, recorded from E4B agent runs on 101 tasks in 69 held-out repositories of the companion study's earlier sets, each with its prompt, the *prefix* the model wrote before the old text, the recorded outcome and the file's content. A frozen-prefix replay regenerates the old text from identical context. A full-call replay regenerates both texts, which matters because the harness's unescape fallback, which retries a missing old text with escape codes turned into raw characters, changes the new text too. Replaying the base model reproduced 95 of 103 recorded not-found failures and all 213 successes; 4 calls had other outcomes. This checks the reconstruction of these calls, not of arbitrary prompts. To test an edit, a user regenerates the calls with the edited model and its random edits, scores them with the harness's matching code, and runs the study kit's gate, which reports escape artefacts, acceptance (the share of old texts found), lost working calls and compile failures.

7. Related resources

EasyEdit gathers knowledge-editing methods behind one interface [9]; TransformerLens, nnsight and pyvene give access to model internals [10–12]. Liu et al. find ground-truth specificity metrics insensitive and propose continuous deviation measures, without random-edit controls [13]. `weightprog` is narrower: a dense-Gemma library that a coding agent drives, with closed-form constructors and comparisons against random edits. We have run no head-to-head comparison.

8. Limits

Real-model validation covers E4B only; mixture-of-experts layers, as in Gemma 4's 26B A4B model, are unsupported, and most methods are tested only on tiny models. The new-target comparison is developer-led, with one run per arm.

9. Conclusion

An edit that fixes one behaviour has to be judged on what it breaks. `weightprog`, OOD-C and the replay set let a coding agent build such edits for Gemma 4 and test them against random edits of the same size, on single tool calls and on a whole agent. In a first test, edits made with and without the library reduced target errors but also changed unrelated outputs, as random edits often did, so preservation limits need calibrating before a larger test of whether the library helps.

10. Availability and licences

- **`weightprog`**: 0.2.1 (tag `v0.2.1`; same code as 0.2.0), Apache-2.0, github.com/ozgung/weightprog. Install with `pip install -e ".[mlx]"` or `".[torch]"`.
- OOD-C: kaggle.com/datasets/ozgung/g4-oodc, run with the bundled `run_ood45.py`; task metadata CC-BY-4.0, code and wheels under their own licences.
- Replay set and study kit: kaggle.com/datasets/ozgung/g4-weight-edit-replay and github.com/ozgung/gemma4-weight-edit-study. To reproduce the gate: `WEIGHTPROG_GATE_DIR=REPORTS WEIGHTPROG_REPLAY_DIR=REPLAY pytest -q weightprog_study/tests/test_gate_regression.py`.
- Notebook: What a weight edit does in Gemma 4 E4B (Section 4).
- Companion study: Paper 1 writeup (reference 2).

References

1. Takehi, Teoh, Zhang, Khattab. Sparks of Weight-Space Programming. Blog post, 2026.
2. Genc. Frontier Models as Weight Editors for Gemma 4 E4B. Companion Writeup, 2026.
3. Meng et al. Locating and Editing Factual Associations in GPT. NeurIPS 2022.
4. Meng et al. Mass-Editing Memory in a Transformer. ICLR 2023.
5. Fang et al. AlphaEdit: Null-Space Constrained Knowledge Editing for Language Models. ICLR 2025.
6. Hu et al. LoRA: Low-Rank Adaptation of Large Language Models. ICLR 2022.
7. Badertdinov et al. SWE-rebench. NeurIPS 2025.
8. Badertdinov et al. SWE-rebench V2. 2026.
9. Wang et al. EasyEdit: An Easy-to-use Knowledge Editing Framework for Large Language Models. ACL 2024 (demo).
10. Nanda, Bloom. TransformerLens. 2022.
11. Fiotto-Kaufman et al. NNsight and NDIF. 2024.
12. Wu et al. pyvene: A Library for Understanding and Improving PyTorch Models via Interventions. NAACL 2024 (demo).
13. Liu et al. Are We Evaluating the Edit Locality of LLM Model Editing Properly?. arXiv preprint, 2026.
14. Alain, Bengio. Understanding intermediate layers using linear classifier probes. arXiv preprint, 2016.
15. nostalgebraist. interpreting GPT: the logit lens. LessWrong, 2020.
16. Elhage et al. A Mathematical Framework for Transformer Circuits. Transformer Circuits Thread, 2021.
17. Zou et al. Representation Engineering: A Top-Down Approach to AI Transparency. arXiv preprint, 2023.
18. Vig et al. Causal Mediation Analysis for Interpreting Neural NLP: The Case of Gender Bias. arXiv preprint, 2020.
19. Olsson et al. In-context Learning and Induction Heads. Transformer Circuits Thread, 2022.
20. Bricken et al. Towards Monosemanticity: Decomposing Language Models With Dictionary Learning. Transformer Circuits Thread, 2023.
21. Huang et al. Transformer-Patcher: One Mistake Worth One Neuron. ICLR 2023.
22. Dai et al. Knowledge Neurons in Pretrained Transformers. ACL 2022.
23. Arditi et al. Refusal in Language Models Is Mediated by a Single Direction. NeurIPS 2024.
24. Turner et al. Steering Language Models With Activation Engineering. arXiv preprint, 2023.
25. Ilharco et al. Editing Models with Task Arithmetic. ICLR 2023.